Neural Networks Fundamentals

AI Accelerator Program

Session 17 | 4:30 PM – 6:30

Instructor;
Mahi Aminu Aliyu
AI & SI Associate AHFID

Opening Story: The Brain Connection 🧠

Imagine you're in a crowded Lagos market — Balogun Market on a Saturday afternoon.

Hundreds of voices, music blaring, hawkers shouting… pure chaos.

Then suddenly, you hear **your friend's voice** calling your name.

How did your brain do that?

How Your Brain Recognized That Voice

Your brain didn't analyze every sound consciously.

Instead, **millions of neurons** in your auditory cortex:

- Detected patterns (pitch, tone, rhythm)
- Fired connections that had been **strengthened through repetition**
- Instantly said: "That's Chidi!"

This is biological learning – and it's exactly what neural networks mimic.

From Biological to Artificial Neurons

Biological Neuron	Artificial Neuron
Dendrites receive signals	Inputs (x_1, x_2, ..., x_n)
Cell body sums signals	Weighted sum + bias
Fires if threshold reached	Activation function
Axon sends output	Output value

Neural networks: **Digital brains learning patterns from data**

Learning Objectives

By the end of today, you will:

1. ✅ Understand neural network architecture (layers, neurons, connections)
2. ✅ Grasp forward propagation (how predictions are made)
3. ✅ Get intuition for backpropagation (how learning happens)
4. ✅ Set up TensorFlow/Keras and train your first neural network
5. ✅ Apply NNs to Nigerian telco churn prediction

Today's Nigerian Business Case 🇳🇬

Nigerian Telco Customer Churn

A telecom company is losing customers to competitors.

- Acquiring new customers costs **₦15,000** (marketing + incentives)
- Retaining existing customers costs **₦2,000** (loyalty offers)
- Monthly revenue per customer: **₦8,500** average

Business Question: Can we predict which customers will churn *before they leave*?

The Dataset: Nigerian Telco Churn

Features:

- Customer demographics: `age`, `state`
- Service details: `plan_type`, `monthly_fee_ngn`, `tenure_months`
- Usage patterns: `data_usage_gb`, `call_minutes`
- Engagement: `customer_support_calls`, `autopay`

Target: `churned` (0 = retained, 1 = left)

Challenge: Patterns are complex — traditional ML struggles. Let's try neural networks!

Part 1: Neural Network Building Blocks

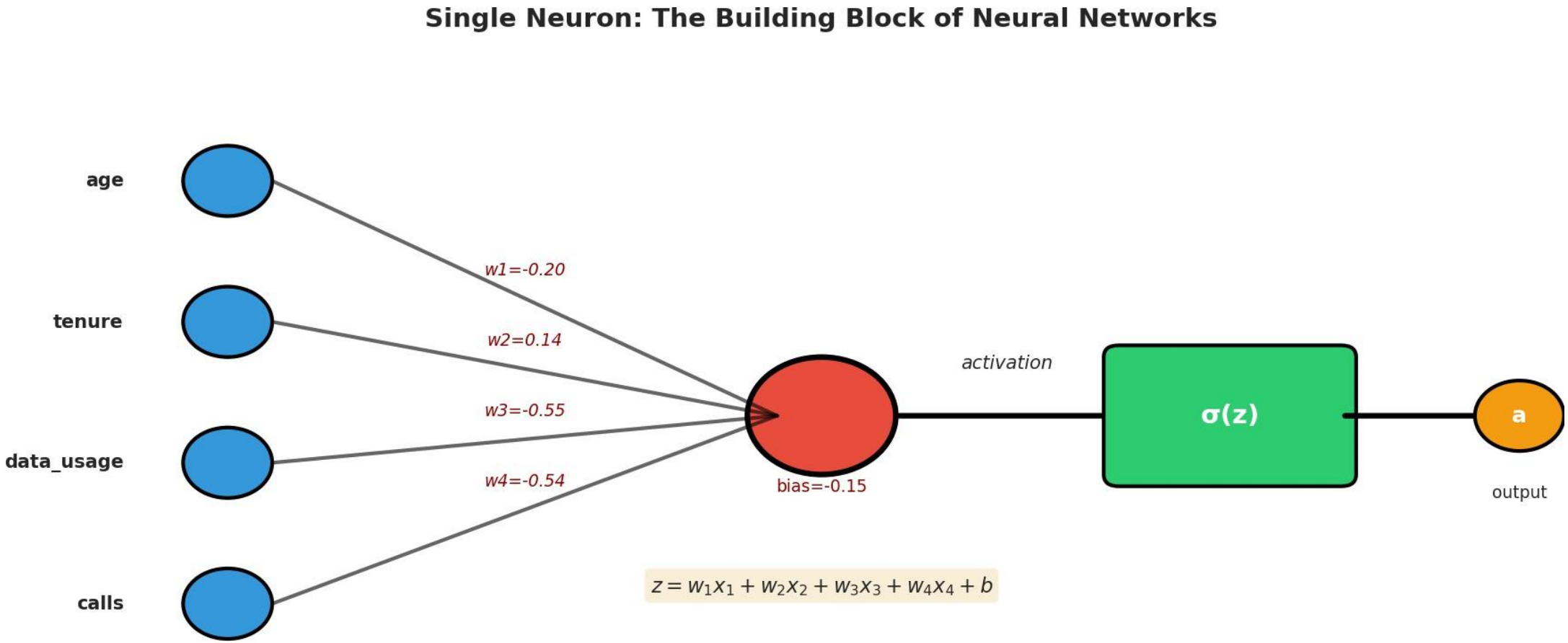
What is a Neural Network?

A **composition of simple mathematical functions** organized in layers:

1. **Weighted sums** (linear combinations)
2. **Nonlinear activations** (introduce complexity)
3. **Stacked layers** (learn hierarchical patterns)

Think of it as **layers of pattern detectors**, each building on the previous layer's insights.

The Single Neuron: Foundation of Everything



The Math (Light Version!)

Step 1: Linear combination

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

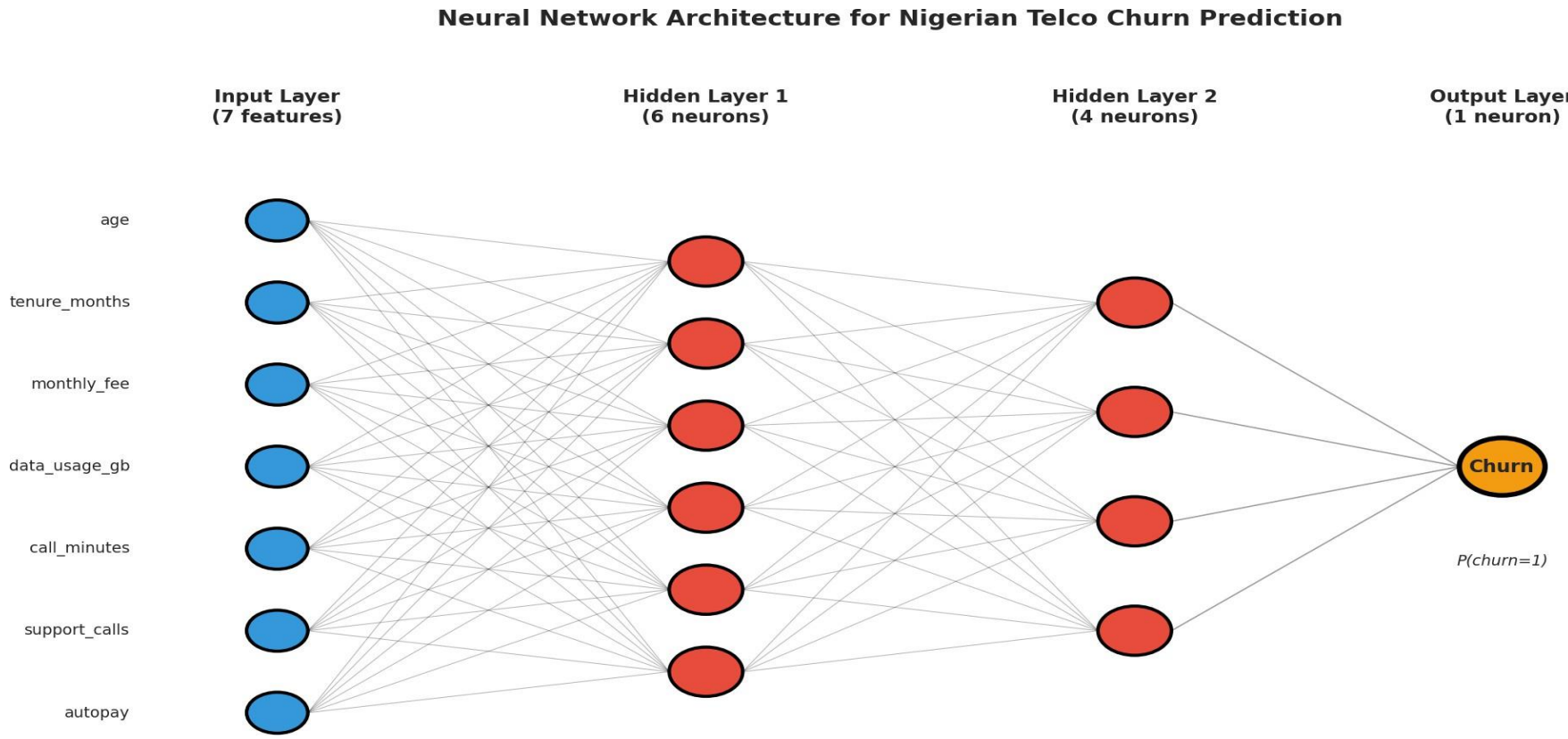
Step 2: Activation function

$$a = \sigma(z)$$

Example: For Nigerian churn customer

- $x_1 = 29$ (age), $x_2 = 18$ (tenure), $x_3 = 8500$ (monthly fee)
- If $w_1 = -0.5, w_2 = 0.8, w_3 = 0.0002, b = 1.2$
- Then $z = -0.5(29) + 0.8(18) + 0.0002(8500) + 1.2 = 3.95$

Multi-Layer Architecture



What Each Layer Learns

Input Layer: Raw features (age, tenure, fee, usage...)

Hidden Layer 1: Simple patterns

- "High support calls = problem"
- "Low tenure + prepaid = risky"

Hidden Layer 2: Complex combinations

- "Young + short tenure + high calls = likely to churn"

Output Layer: Final prediction

- P(churn = 1) = 0.73 → High risk customer!

Why Multiple Layers? (Depth Matters)

Shallow network (1 hidden layer):

- Learns simple patterns
- Good for basic problems

Deep network (2-3+ hidden layers):

- Learns hierarchical features
- Better for complex patterns
- More parameters to tune

For Nigerian churn: 2 hidden layers strike good balance between power and simplicity

Part 2: Activation Functions

Why Do We Need Activation Functions?

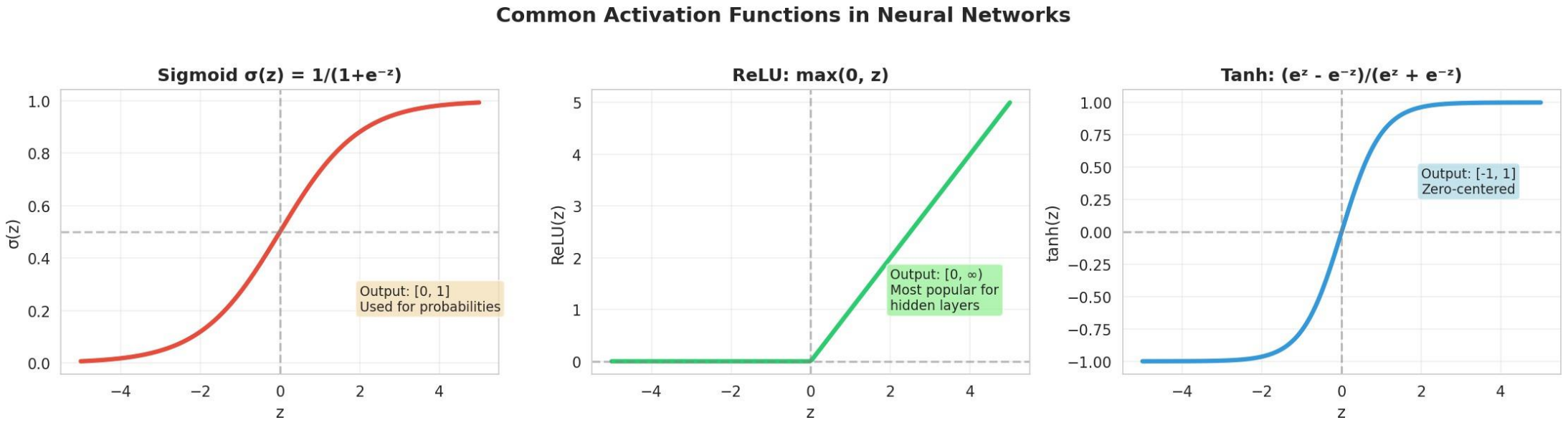
Without activation functions:

- Network is just stacked linear equations
- $z^{(2)} = W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)}$ = Still just one linear function!
- Can only learn linear patterns (like basic linear regression)

With activation functions:

- Introduce **nonlinearity**
- Can learn complex patterns (curves, interactions, thresholds)

The Three Most Common Activations



Each has different properties and use cases!

Sigmoid: $\sigma(z) = 1/(1 + e^{-z})$

Properties:

- Output range: [0, 1]
- Smooth S-curve
- Interpretable as probability

Use cases:

- ✅ **Output layer** for binary classification (churn prediction)
- ❌ Hidden layers (vanishing gradient problem)

Nigerian churn example: Output 0.73 means 73% probability of churning

ReLU: max(0, z)

Properties:

- Output range: [0, ∞)
- Super simple: if z > 0, output z; else 0
- Fast to compute

Use cases:

- ✅ **Hidden layers** (most popular choice!)
- Solves vanishing gradient problem
- Sparse activation (many zeros = efficient)

Why it's dominant: Simple + effective = winner

Tanh: $(e^z - e^{-z})/(e^z + e^{-z})$

Properties:

- Output range: [-1, 1]
- Zero-centered (unlike sigmoid)
- Similar shape to sigmoid

Use cases:

- Hidden layers when you need negative values
- Less common than ReLU nowadays

When to use: Specific problems where zero-centering helps

Choosing Activation Functions: Rule of Thumb

Layer Type	Recommended Activation	Why?
Hidden layers	ReLU	Fast, effective, prevents vanishing gradients
Output (binary)	Sigmoid	Outputs probability [0,1]
Output (multi-class)	Softmax	Outputs probability distribution
Output (regression)	None (linear)	Can output any real number

For Nigerian churn: ReLU in hidden layers, Sigmoid in output

Part 3: Forward Propagation

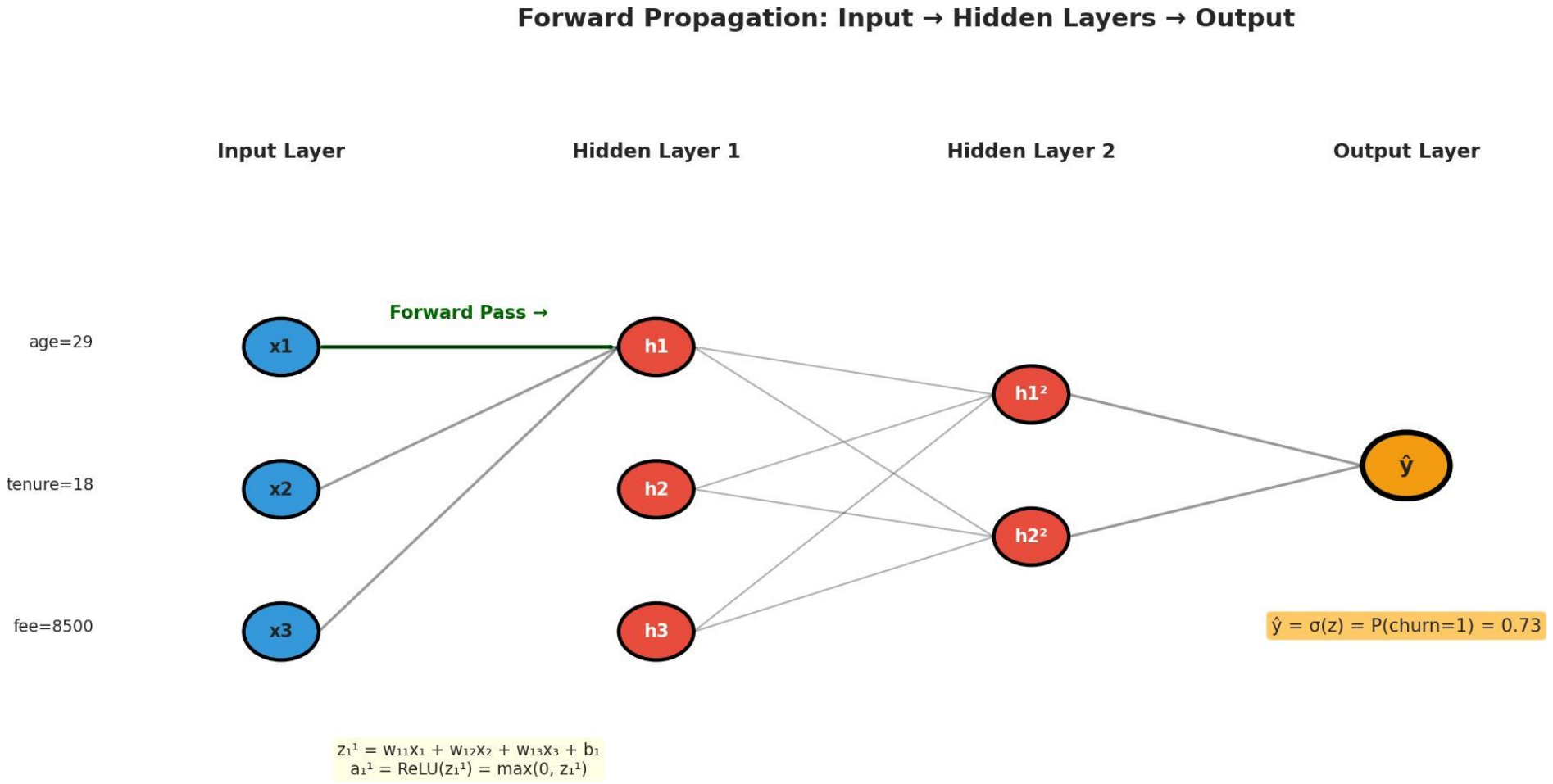
Forward Propagation: Making Predictions

The journey from input to output:

1. Start with customer data (age, tenure, fee...)
2. Multiply by weights, add bias → get z
3. Apply activation → get a
4. Repeat for each layer
5. Final output = prediction!

It's called "forward" because we move left-to-right through the network

Forward Propagation Visualization



Forward Pass: Step-by-Step Example

Input: Nigerian customer (age=29, tenure=18, fee=8500)

Layer 1 (Hidden):

- Neuron 1: $z_1^{(1)} = w_{11}(29) + w_{12}(18) + w_{13}(8500) + b_1$
- Apply ReLU: $a_1^{(1)} = max(0, z_1^{(1)})$
- Repeat for all 6 neurons in layer 1

Layer 2 (Hidden):

- Use outputs from Layer 1 as inputs
- Repeat weighted sum + ReLU

Output:

- Final weighted sum + Sigmoid = 0.73 (73% churn probability)

Intuition: Each Layer Transforms the Data

Layer 1: Takes raw features → detects basic patterns

Layer 2: Takes basic patterns → combines into complex patterns

Output: Takes complex patterns → makes final decision

Think of it like: Raw ingredients → chopped → cooked → plated meal

Each transformation makes the problem easier for the next layer!

Part 4: Learning (Backpropagation Intuition)

The Learning Problem

We have:

- Network architecture (layers, neurons)
- Random initial weights

We need:

- Weights that make accurate predictions

How? Compare predictions to truth, measure error, adjust weights to reduce error

This process is called backpropagation +gradient descent

Loss Function: Measuring Error

Binary Cross-Entropy Loss (for classification):

$$L = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Don't memorize the formula! Just understand:

- When prediction $\hat{y}$ is close to true label y → **Loss is small** ✅
- When prediction is wrong → **Loss is large** ❌

Goal: Find weights that minimize loss across all customers

Gradient Descent: The Optimization Algorithm

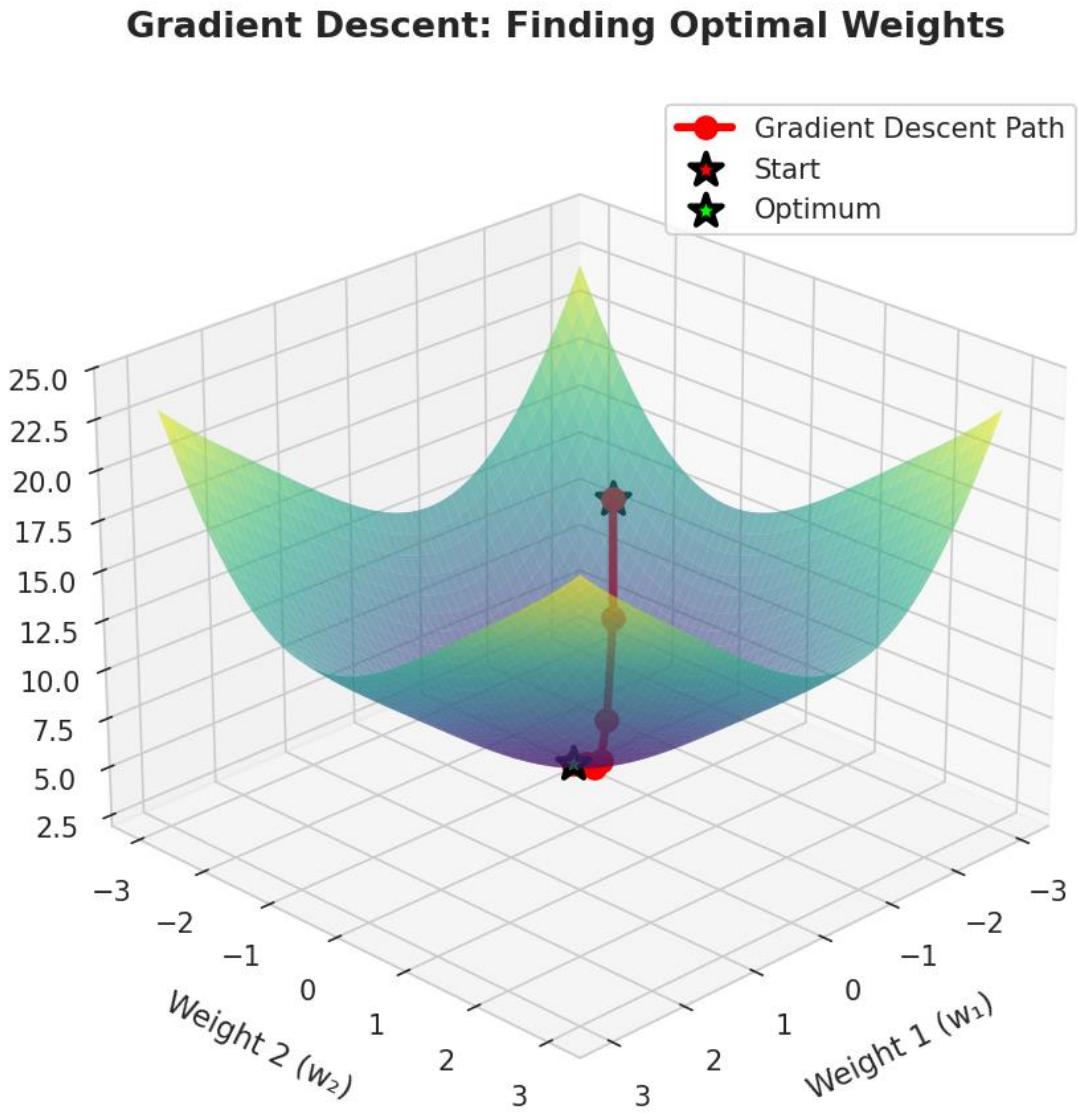
Intuition: You're hiking in fog, trying to reach the valley (minimum loss)

1. Check which direction is downhill (compute gradient)
2. Take a step in that direction (update weights)
3. Repeat until you reach the bottom

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

- η = **learning rate** (step size)
- $\frac{\partial L}{\partial w}$ = **gradient** (which direction to go)

Gradient Descent Visualization



Backpropagation: Computing Gradients Efficiently

The challenge: Network has thousands of weights. How do we compute $\frac{\partial L}{\partial w}$ for each one?

The solution: Chain rule from calculus!

Start at output → work backward through layers → compute how each weight contributed to error

Why "back"propagation: We propagate error backward through the network

Backpropagation Intuition (No Math!)

Think of it like debugging code:

1. **Output layer:** "I predicted 0.73, truth was 1.0 — I'm off by 0.27"
2. **Hidden layer 2:** "Which of my neurons contributed most to that error?"
3. **Hidden layer 1:** "Which of MY inputs made those neurons wrong?"
4. **Weights:** "Adjust me to fix those mistakes!"

Each layer asks: "How much am I responsible for the final error?"

3
4

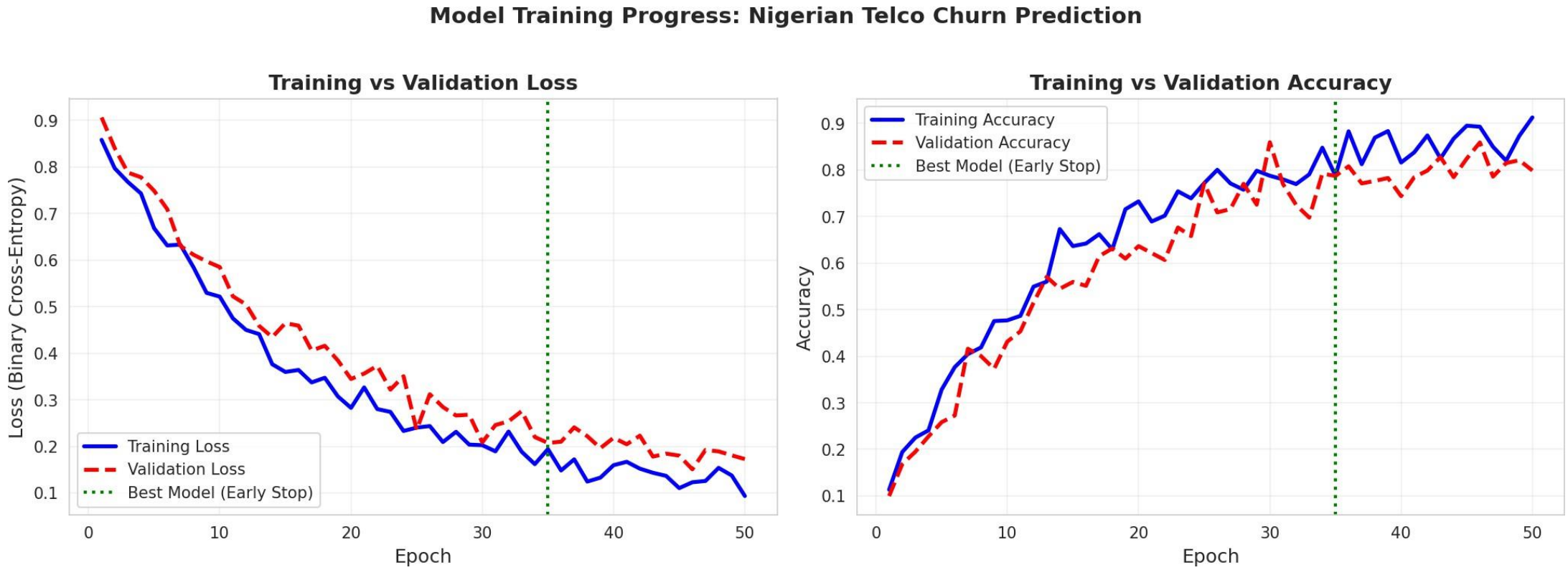
The Training Loop

```
for epoch in range(num_epochs):
    # 1. Forward pass: Make predictions
    predictions = model.forward(X_train)

    # 2. Compute loss
    loss = binary_crossentropy(y_train, predictions)

    # 3. Backward pass: Compute gradients
    gradients = model.backward(loss)

    # 4. Update weights
    model.update_weights(gradients, learning_rate)
```

Repeat thousands of times → Network learns patterns!

Part 5: Training Dynamics

Training Curves: Monitoring Learning



What we want to see:

- Training loss ↓ (learning)
- Validation loss (generalizing)

Key Training Concepts

- Epoch: One complete pass through entire training dataset
- Batch: Subset of data used in one gradient update
  - Full batch: Use all data (slow)
  - Mini-batch: Use 32-256 samples (most common)
  - Stochastic: Use 1 sample (noisy)
- Iteration: One gradient update (one batch processed)
- Example: 1000 samples, batch size 32 → 31 iterations per epoch

Early Stopping: Preventing Overfitting

- The problem:
  - Training loss keeps decreasing
  - But validation loss starts increasing (overfitting!)
- The solution: Early stopping
  - Monitor validation loss
  - Stop training when it stops improving
  - Use best model from training
- Green line in previous slide: Optimal stopping point (epoch 35)

Regularization Techniques

Dropout: Randomly "turn off" neurons during training

- Forces network to learn redundant representations
- Reduces overfitting
- Typical value: 0.2-0.5 (20-50% dropout)

L2 Regularization: Penalize large weights

- Add $\lambda \sum w^2$ to loss function
- Encourages smaller, more generalized weights

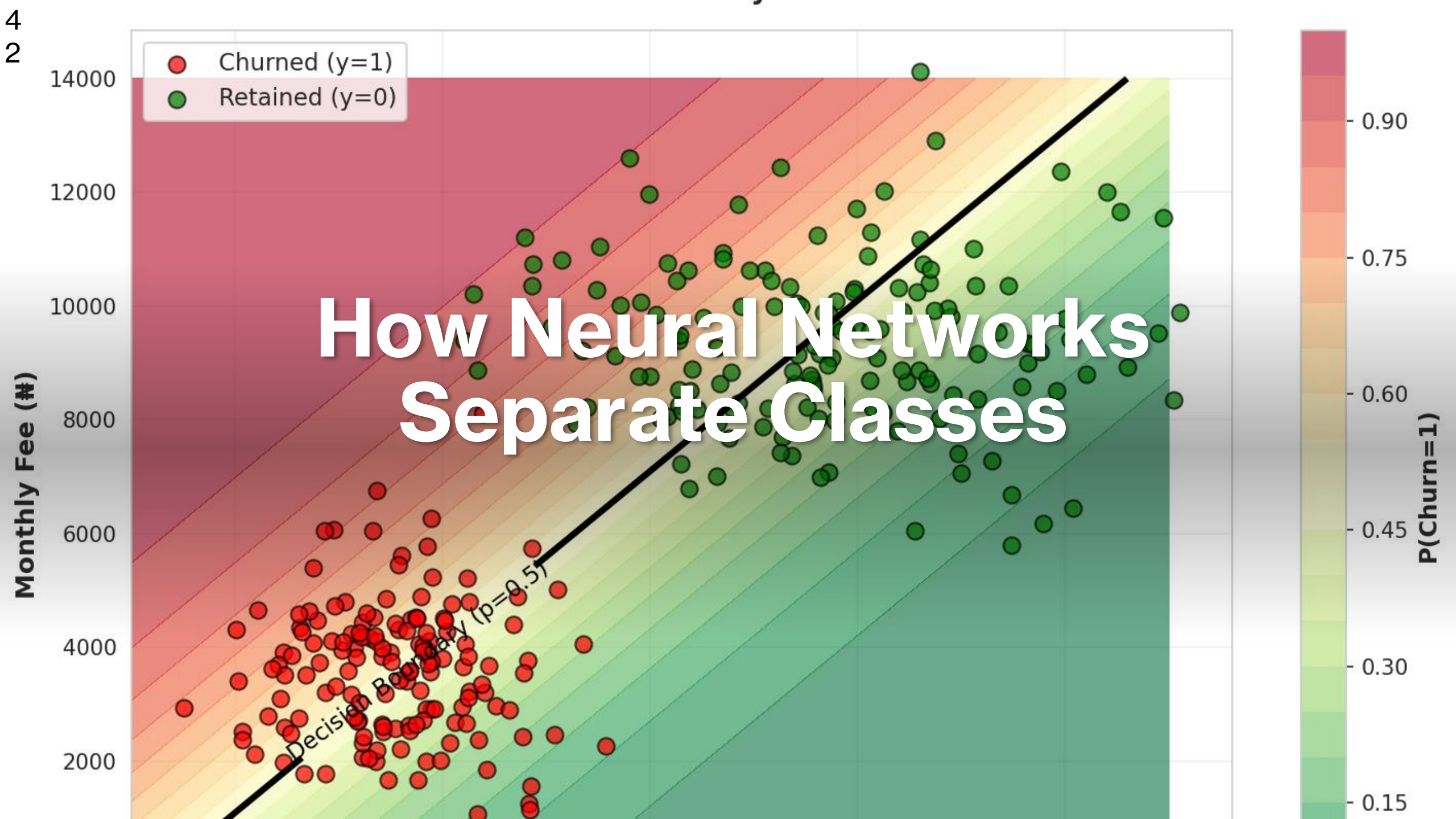
For Nigerian churn: Start with dropout=0.3 in hidden layers

Hyperparameters to Tune

Hyperparameter	What it controls	Typical values
Learning rate	Step size in gradient descent	0.001 - 0.01
Batch size	Samples per gradient update	32, 64, 128
Epochs	Training iterations	50-200 (with early stop)
Hidden layers	Network depth	2-3 for tabular data
Neurons per layer	Network width	32, 64, 128
Dropout rate	Regularization strength	0.2-0.5

Start simple, increase complexity only if needed!

Part 6: Decision Boundaries


How Neural Networks Separate Classes
Churned (y=1)
Retained (y=0)
Decision Boundary (p=0.5)
Monthly Fee (₦)
14000
12000
10000
8000
6000
4000
2000
P(Churn=1)
0.90
0.75
0.60
0.45
0.30
0.15

Why Neural Networks Excel

Traditional ML (Logistic Regression, SVM):

- Learn relatively simple boundaries
- Require manual feature engineering
- Good for linearly separable problems

Neural Networks:

- Learn complex, curved boundaries
- Automatically discover useful features
- Excel when patterns are intricate

Nigerian churn: Interactions between age, tenure, usage → NN captures these naturally!

Part 7: When to Use Neural Networks

Neural Networks vs Classical ML

Use Neural Networks when:

- ✅ Unstructured data (images, text, audio)
- ✅ Large datasets (10,000+ samples)
- ✅ Complex patterns and interactions
- ✅ You have GPU resources

Use Classical ML (Random Forest, XGBoost) when:

- ✅ Small/medium tabular data
- ✅ Need interpretability
- ✅ Limited compute resources
- ✅ Fast training required

Nigerian Telco Churn: NN or Not?

Dataset: ~1000 customers, 10 features, tabular

Verdict: Either could work!

- Classical ML: Might be sufficient, more interpretable
- Neural Networks: Good learning exercise, slightly better performance

Real-world recommendation:

- Start with Random Forest/XGBoost
- Try NN if you need that extra 2-3% accuracy
- Ensemble both for production systems

Our goal today: Learn NNs with practical data, regardless of "optimal" choice

Part 8: Cost-Benefit Analysis

The Business Case: Cost of Errors

Scenario	Cost	Impact
False Negative (predict stay, actually churns)	₦15,000	Must acquire new customer
False Positive (predict churn, actually stays)	₦2,000	Unnecessary retention offer
True Positive (correctly predict churn)	₦2,000	Proactive retention (saved!)
True Negative (correctly predict stay)	₦0	No action needed

Implication: We care MORE about recall (catching churners) than precision!

Optimizing for Business Value

Adjust decision threshold

- Default: Predict churn if P(churn) > 0.5
- Business-aware: Predict churn if P(churn) > 0.3

Why lower threshold?

- Catch more churners (higher recall)
- Accept more false positives (₦2k vs ₦15k — worth it!)

This is where ML meets business strategy!

5
0

Feature Importance for Retention Strategy

Top predictors of churn:

1. **Low tenure** (<12 months) → Improve onboarding experience
2. **High support calls** (>3) → Fix service quality issues
3. **Prepaid plans** → Offer postpaid conversion incentives
4. **Low data usage** → Not engaged, offer better plans
5. **No autopay** → Friction in payment → Encourage autopay

NN helps identify patterns → Business acts on insights

Part 9: Environment Setup (TensorFlow/Keras)

TensorFlow + Keras

TensorFlow: Google's deep learning framework (low-level, powerful)

Keras: High-level API on top of TensorFlow (user-friendly)

Why Keras?

- Simple, intuitive syntax
- Great for beginners and prototyping
- Production-ready (used by Uber, Netflix, etc.)

Installation:

```
pip install tensorflow
```

The Keras Workflow

```
# 1. Define model architecture
model = Sequential([
    Dense(64, activation='relu', input_shape=(10,)),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

# 2. Compile (specify loss, optimizer, metrics)
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# 3. Train
model.fit(X_train, y_train, epochs=50,
          validation_split=0.2  batch_size=32
```

NumPy Implementation (Optional Deep Dive)

Why learn NumPy implementation?

- Understand what's happening "under the hood"
- Builds true intuition for forward/backward passes
- Helps debug issues in real models

Then switch to Keras:

- Much faster to prototype
- Production-ready
- GPU acceleration automatic

Best learning path: NumPy intuition first → Keras for practice

Summary & Key Takeaways

5
6

What We Learned Today

1. ✅ **Neural network architecture:** Layers, neurons, weights, biases
2. ✅ **Activation functions:** ReLU for hidden, Sigmoid for output
3. ✅ **Forward propagation:** Input → hidden layers → output (prediction)
4. ✅ **Backpropagation intuition:** Computing gradients to update weights
5. ✅ **Gradient descent:** Iteratively minimizing loss
6. ✅ **Training dynamics:** Epochs, batches, early stopping, regularization
7. ✅ **Business application:** Nigerian telco churn with cost-sensitive decisions

The Big Picture

Neural networks are:

- Universal function approximators (can learn any pattern)
- Layers of simple transformations (weighted sums + activations)
- Trained via gradient descent (adjust weights to minimize loss)

Success requires:

- Good data (garbage in = garbage out)
- Proper architecture (depth, width, activations)
- Careful training (learning rate, epochs, regularization)
- Business context (cost-sensitive thresholds, interpretability)

Practice Exercises & Assignment

Practice Exercises (3 exercises):

- Build simple 2-layer network from scratch (NumPy)
- Experiment with different activations
- Train with Keras on Nigerian churn
- Tune hyperparameters
- Visualize training curves
- Evaluate with confusion matrix

Assignment (100 points):

- Full churn prediction pipeline
- Architecture experiments
- Cost-sensitive evaluation

Next Week: Convolutional Neural Networks (CNNs)

Module 3 continues with Computer Vision:

- Image data and convolutions
- CNN architectures (filters, pooling)
- Transfer learning
- Nigerian use case: Crop disease detection

Get excited! We're moving from tabular → image data 📸

Q&A

Questions?

Remember: Neural networks are just **pattern-learning machines**.

Start simple, iterate, and always connect to business value!

See you in the practice session! 🚀